UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Faculty of Engineering, Built Environment and Information Technology

EBB 320

Control Systems

Practical 3: Report

Group 28

Name and Surname	Student Number	Signature	% Contribution
A. van Niekerk	19007583		33
TS Fogwill	18321692		33
KC. Mattheus	17049271		33

By signing this assignment we confirm that we have read and are aware of the University of Pretoria's policy on academic dishonesty and plagiarism and we declare that the work submitted in this assignment is our own as delimited by the mentioned policies. We explicitly declare that no parts of this assignment have been copied from current or previous students' work or any other sources (including the internet), whether copyrighted or not. We understand that we will be subjected to disciplinary actions should it be found that the work we submit here does not comply with the said policies.

November 9, 2021

Contents

1 Work Breakdown . . . 2
1.1 Adriaan van Niekerk . . . 2
1.2 Timothy Fogwill . . . 2
1.3 Kalvin Mattheus . . . 2
2 Introduction and Aim . . . 3
2.1 Introduction . . . 3
2.2 Aim . . . 3
3 Controller Design . . . 3
3.1 PID Discretization . . . 3
3.2 Motor subsystem controller . . . 4
3.3 State-control subsystem controller . . . 12
4 Simulations . . . 13
4.1 Motor subsystem simulations . . . 13
5 Results . . . 17
5.1 Line-sensor subsystem results . . . 17
5.2 SC^3 subsystem results . . . 18
5.3 Motor subsystem results . . . 19
6 Discussion . . . 20
7 Conclusion . . . 21

A Motor Control Python Code **24**

B Motor Control Arduino code **27**

C Controller design and simulation python code **30**

D Line sensor Arduino code **33**

E Line sensor Python code **35**

List of Figures

3.2.1 Open-loop time response of the motor plant, $G_M(s)$, to a unit step input 5
3.2.2 Tuning parameters obtained from the open-loop time response of the motor plant, $G_M(s)$, to a unit step input 8
3.2.3 Closed loop time response of motor plant with ultimate controller gain of 0.09812 11
4.1.1 Simulation of the closed loop response to a step input using the controller approximated with SIMC 13
4.1.2 Simulation of the closed loop response to a step input using the controller approximated with Cohen-Coon 15
4.1.3 Simulation of the closed loop response to a step input using the controller approximated with Ziegler-Nichols 16
5.1.1 Line offset vs time plot of the sensor subsystem reading the generated track 18
5.2.1 SC^3 PID Tuning 18
5.2.2 SC3 PID Practical Tuning 19
5.3.1 Motor speed simulation with no PID controller 19
5.3.2 Motor speed simulation using SIMC PID controller 20
5.3.3 Motor speed simulation using Cohen-Coon PID controller 20
5.3.4 Motor speed simulation using Ziegler$_{N}icolsPIDcontroller$ 20

1 Work Breakdown

1.1 Adriaan van Niekerk

- Designed, built, simulated and tested the new motor subsystem(practical's 2 motor system broke).
- Determined constants from motor simulation.
- Realized motor PID controllers and generated results from these.
- Conducted language, grammar and format editing.

1.2 Timothy Fogwill

- Wrote the aim and introduction sections of the report.
- Controller design and report sections for PI tuning using SIMC, Cohen-Coon process reaction curve and Ziegler-Nichols oscillation method.
- Simulated the controllers that were designed using the PI tuning using SIMC method, Cohen-Coon process reaction curve method and the Ziegler-Nichols oscillation method, wrote the report for these sections.
- Wrote the line-sensor subsystem report section and built, simulated and tested the line-sensor subsystem.
- Wrote the report's discussion.
- Conducted language, grammar and format editing.

1.3 Kalvin Mattheus

- Designed, built, simulated and tested the S^3C subsystem.
- Wrote the PID discretization section of the report.
- Wrote the S^3C subsystem simulation and results sections of the report.
- Conducted language, grammar and format editing.

2 Introduction and Aim

2.1 Introduction

PID controllers are designed, simulated and realized using 3 separate methods, namely the Cohen-Coon process reaction curve method, the Ziegler-Nichols oscillation method and the PI tuning using SIMC method. These controllers are analyzed and discussed and the most appropriate one for the context of a line-following ARV is chosen. A separate controller is designed, simulated and realized to control the S^3C subsystem. This controller, in conjunction with the chosen motor subsystem controller and the line sensor subsystem, make up the integrated ARV that can follow a coloured line with minimal error.

2.2 Aim

The aim of this practical is to design PID controllers for the motor and S^3C subsystems of an ARV. These controllers are to be designed with the purpose of minimizing the error between the ARV position and a line position.

3 Controller Design

3.1 PID Discretization

For the PID controller to work on a micro controller the PID functions need to be descretized. The time domain PID controller function is as follows:

$$pwm_{\%} = pwm_b + K_c \left(e(t) + \frac{1}{\tau_I} \int_0^t e(t^*)\, dt^* + \tau_D \frac{de(t)}{dt} \right) \tag{1.1}$$

Where e is the error sent by the SC^3 subsystem and $pwm_{\%}$ is the output determined by the controller.

Both sides are differentiated to achieve discretization:

$$\frac{dpwm_{\%}(t)}{dt} = K_c \frac{de(t)}{dt} + \frac{K_c}{\tau_I} e(t) + K_c \tau_D \frac{d^2 e(t)}{dt^2} \tag{1.2}$$

Using Backward differentiation on $\frac{dpwm_{\%}(t)}{dt}$, $\frac{de(t)}{dt}$ and $\frac{d^2e(t)}{dt^2}$ results in the following:

$$\begin{aligned} \frac{pwm_{\%}(t_n) - pwm_{\%}(t_{n-1})}{h} &= K_c \frac{e(t_n) - e(t_{n-1})}{h} + \frac{K_c}{\tau_I} e(t) \\ &\quad + K_c \tau_D \frac{\frac{de(t_n)}{dt_n} - \frac{de(t_{n-1})}{dt_{n-1}}}{h} \end{aligned} \tag{1.3}$$

Using Backward differentiation on $\frac{de(t_n)}{dt_n}$ and $\frac{de(t_{n-1})}{dt_{n-1}}$ results in the following:

$$\frac{pwm_{\%}(t_n) - pwm_{\%}(t_{n-1})}{h} = K_c \frac{e(t_n) - e(t_{n-1})}{h} + \frac{K_c}{\tau_I} e(t) + K_c \tau_D \frac{\frac{e(t_n)-e(t_{n-1})}{h} - \frac{e(t_{n-1})-e(t_{n-2})}{h}}{h} \tag{1.4}$$

Solving for $pwm_{\%}(t)$ from above results in the discrete-time PID controller function that will be used in the micro controller:

$$pwm_{\%}(t_n) = pwm_{\%}(t_{n-1}) + K_c \left[e(t_n) - e(t_{n-1})\right] + \frac{K_c h}{\tau_I} e(t_n) + \frac{K_c \tau_D}{h} \left[e(t_n) - 2e(t_{n-1}) + e(t_{n-2})\right] \tag{1.5}$$

3.2 Motor subsystem controller

The PID controller for the motor subsystem has the following form:

$$G_M(s) = \frac{K e^{-t_d s}}{\tau_p s + 1}$$

Using the values obtained for the transfer system from practical 2:

$$K = 81$$

$$t_d = 0.1s$$

$$\tau_p = 0.46s$$

$$G_M(s) = \frac{17e^{-s}}{1.38s + 1}$$

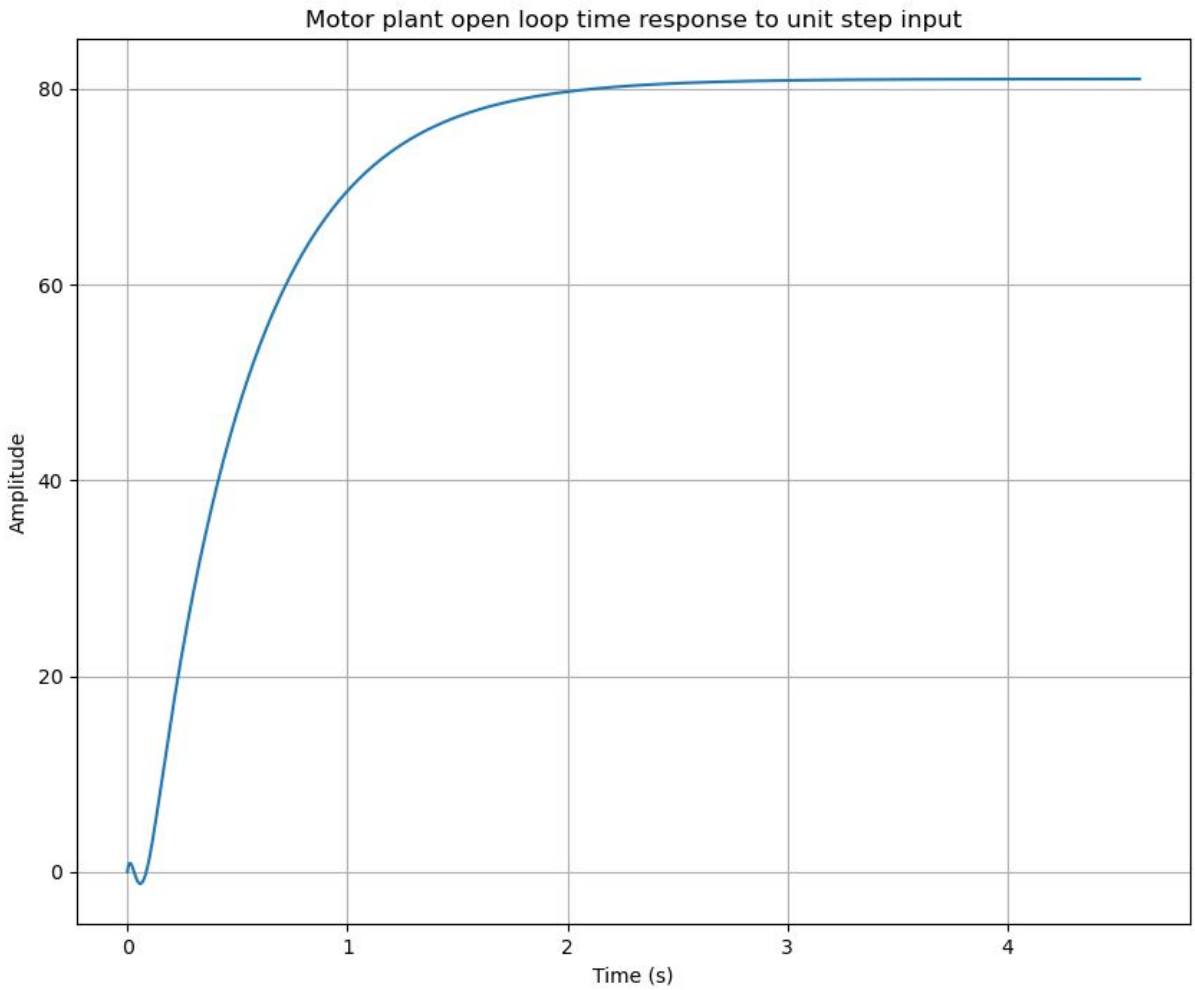

Figure 3.2.1: Open-loop time response of the motor plant, $G_M(s)$, to a unit step input

This PID controller is then modified by a plant controller. $G_C(s)$. The purpose of this controller is to minimize the error between the measured motor speed and the motor setpoint speed. This is accomplished by designing the plant using 3 separate methods: PI tuning using SIMC, Cohen-Coon tuning and the Ziegler-Nichols oscillation method. These methods all create controller with the form:

$$G_c(s) = K_c(1 + \frac{1}{\tau_1 s} + \tau_D s)$$

PI tuning using SIMC

The approximation of the plant has the following form:

$$G_c(s) = K_c(1 + \frac{1}{\tau_1 s} + \tau_D s)$$

These parameters are determined as follows:

$$\tau_D = 0$$

$$\tau_1 = min(\tau_p, 4(\tau_c + t_d))$$
$$K_c = \frac{\tau_p}{K(\tau_c + t_d)}$$

The tuning parameter, τ_c, is generally chosen such that $\tau_c > t_d$. However the scope of this practical requires that τ_c be chosen such that:

$$\tau_c = t_d = 0.1$$

Therefore the parameters of $G_c(s)$ can be calculated as follows:

$$\tau_1 = min(\tau_p, 4(\tau_c + t_d))$$
$$\tau_1 = min(0.46, 8 \times (0.1))$$
$$\tau_1 = min(0.46, 0.8)$$
$$\tau_1 = 0.46$$

$$\tau_D = 0$$

$$K_c = \frac{\tau_p}{K(\tau_c + t_d)}$$
$$K_c = \frac{0.46}{(81)(2 \times 0.1)}$$
$$K_c = \frac{0.46}{16.2}$$
$$K_c = 0.02840$$

Therefore the PID controller of the plant is:

$$G_c(s) = K_c(1 + \frac{1}{\tau_1 s} + \tau_D s)$$
$$G_c(s) = 0.02840 \times (1 + \frac{1}{0.46s})$$
$$G_c(s) = (\frac{1 + 0.46s}{16.20s})$$

Cohen-Coon process reaction curve method

This approximation method works by applying a step input and using the response to said step input to determine the tuning parameters. These tuning parameters are then used to calculate the parameters of the PID controller.

The approximation of the PID controller of the plant has the following form:

$$G_c(s) = K_c(1 + \frac{1}{\tau_1 s} + \tau_D s)$$

With

$$K_c = \frac{v_0}{K_0 \times \tau_0}(\frac{4}{3} + \frac{\tau_0}{4v_0})$$
$$\tau_1 = \frac{\tau_0(32v_0 + 6\tau_0)}{13v_0 + 8\tau_0}$$
$$\tau_D = \frac{4\tau_0 v_0}{11v_0 + 2\tau_0}$$

These parameters are calculated using the tuning parameters v_0, τ_0 and K_0. These are determined using the following formulae:

$$\tau_0 = t_1 - t_0$$
$$v_0 = t_2 - t_1$$
$$K_0 = \frac{y_\infty - y_0}{u_\infty - u_0}$$

where t_0 is the time at which the step input is applied, t_1 is the time at which the system responds to the step input and t_2 is the time where the maximum slope tangent intercepts with the steady-state output value. y is the output of the system and u is the input to said system.

The open-loop time response to a step input is shown below in figure 3.2.1.

This figure can be used to determine the tuning parameters of the PID plant controller. The 3 time values t_0, t_1 and t_2 are shown using red lines. The steady-state value, y_∞ is shown by the horizontal black line.

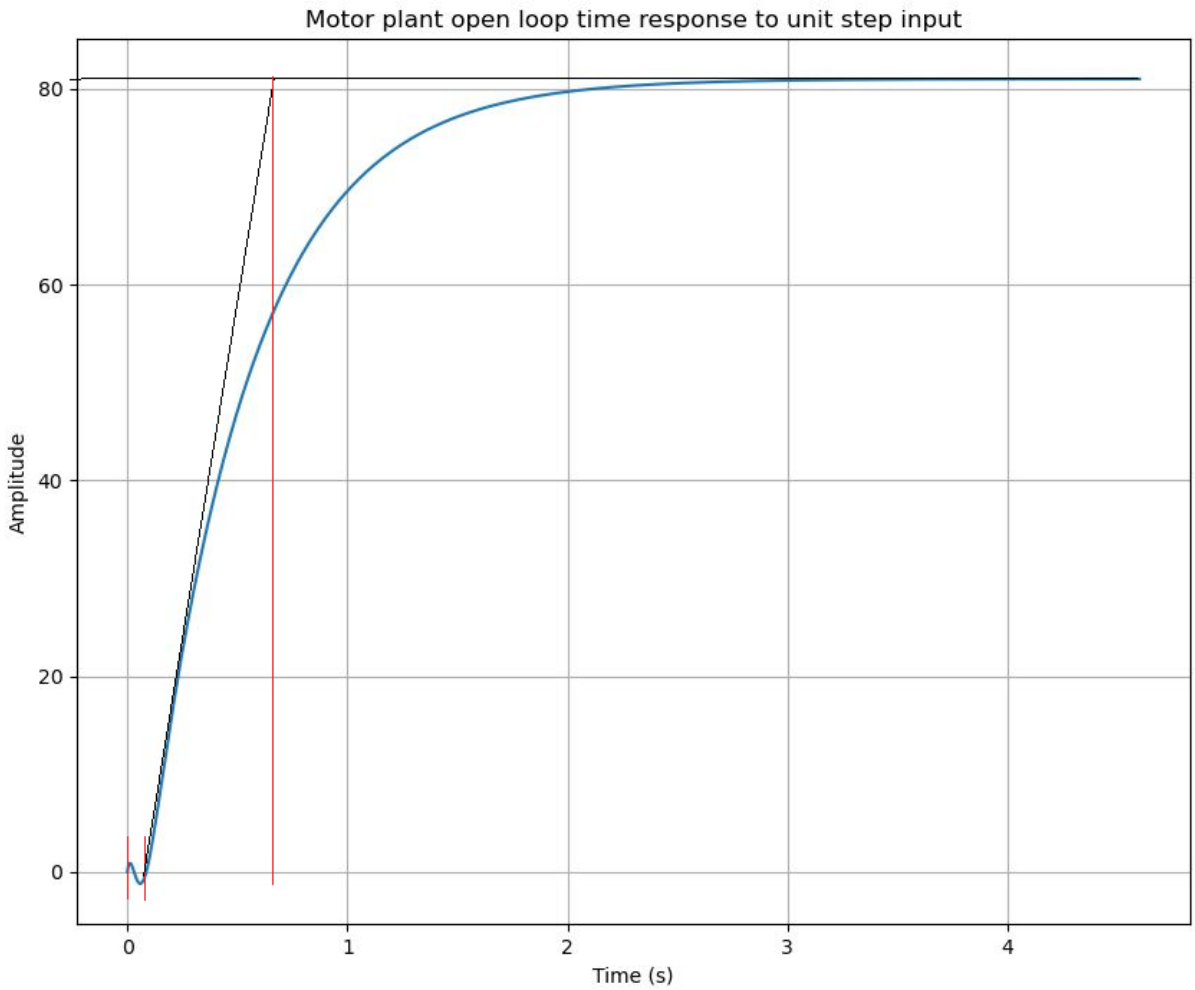

Figure 3.2.2: Tuning parameters obtained from the open-loop time response of the motor plant, $G_M(s)$, to a unit step input

The table below shows the values of the tuning parameters needed to calculate the parameters of the PID controller. These values are obtained by reading figure 3.2.2. The values of the input, u, can be determined by viewing the definition of the unit step function.

Parameter	**Value**
t_0	0s
t_1	0.1s
t_2	0.67s
u_0	0
u_∞	1
y_0	0
y_∞	81

Therefore the tuning parameters can be calculated using the above table's values:

$$\tau_0 = t_1 - t_0$$

$$\tau_0 = 0.1 - 0$$

$$\tau_0 = 0.1$$

$$v_0 = t_2 - t_1$$
$$v_0 = 0.67 - 0.1$$
$$v_0 = 0.567$$

$$K_0 = \frac{y_\infty - y_0}{u_\infty - u_0}$$
$$K_0 = \frac{81 - 0}{1 - 0}$$
$$K_0 = 81$$

Using these tuning parameters, the approximate value of the PID controller's parameters can be calculated:

$$K_c = \frac{v_0}{K_0 \times \tau_0}(\frac{4}{3} + \frac{\tau_0}{4v_0})$$
$$K_c = \frac{0.567}{81 \times 0.1}(\frac{4}{3} + \frac{0.1}{4 \times 0.567})$$
$$K_c = 0.0964$$

$$\tau_1 = \frac{\tau_0(32v_0 + 6\tau_0)}{13v_0 + 8\tau_0}$$
$$\tau_1 = \frac{0.1 \times (32(0.567) + 6(0.1))}{13(0.567) + 8(0.1)}$$
$$\tau_1 = 0.2543$$

$$\tau_D = \frac{4.\tau_0.v_0}{11v_0 + 2\tau_0}$$
$$\tau_D = \frac{4(0.1)(0.567)}{11(0.567) + 2(0.1)}$$
$$\tau_D = 0.0352$$

Therefore the PID controller for the motor plant can be approximated by the following:

$$G_c(s) = K_c(1 + \frac{1}{\tau_1 s} + \tau_D s)$$

$$G_c(s) = 0.0964(1 + \frac{1}{0.2543s} + 0.0352s)$$

Ziegler-Nichols oscillation method

This method for approximating a PID plant controller implementing proportional control of the controller by setting the derivative term, τ_D, to a small value and the integral term, τ_1, to a large value. This means that both responses are minimal (derivative term tends to 0 as τ_D tends to 0 and the integral term tends to 0 as τ_1 tends to ∞). The controller gain is then set to a small, non-zero value. A small set-point change is then introduced and the controller gain is altered until a controller gain is found that produces sustained oscillations with a constant amplitude. This gain and the period between the oscillations that it produces are then used to calculate the parameters of the PID plant controller.

The approximation of the PID controller of the plant has the following form:

$$G_c(s) = K_c(1 + \frac{1}{\tau_1 s} + \tau_D s)$$

These parameters of the PID controller can be determined using the following relationships:

$$K_c = 0.6K_{cu}$$
$$\tau_1 = 0.5P_u$$
$$\tau_D = 0.125P_u$$

where P_u is the period of oscillations and K_{cu} is the gain that produces steady-state, sustained oscillations.

The motor plant is modified by K_{cu} to find the ultimate controller gain where the modified motor plant is stable. Therefore

$$G_{MM}(s) = \frac{K_{cu \times 81e^{-0.1s}}}{0.46s + 1}$$

This figure, figure 3.2.3 is obtained when $K_{cu} = 0.09812$. This gain was obtained by starting at $K_{cu} = 0.001$ and increasing the gain until the direction of the function with a high first derivative changed. At $K_{cu} = 0.001$ the response has a steep in the range on roughly [0, 2.5], at $K_{cu} = 0.1$ the steepest curve is at $t = 30$. The gain was decreased when the first derivative of the previous response was high on the right of the figure, and increased when on the left. This process was repeated until a sufficiently stable response was found. This gain of $K_{cu} = 0.09812$ produces the following oscillating closed loop time response:

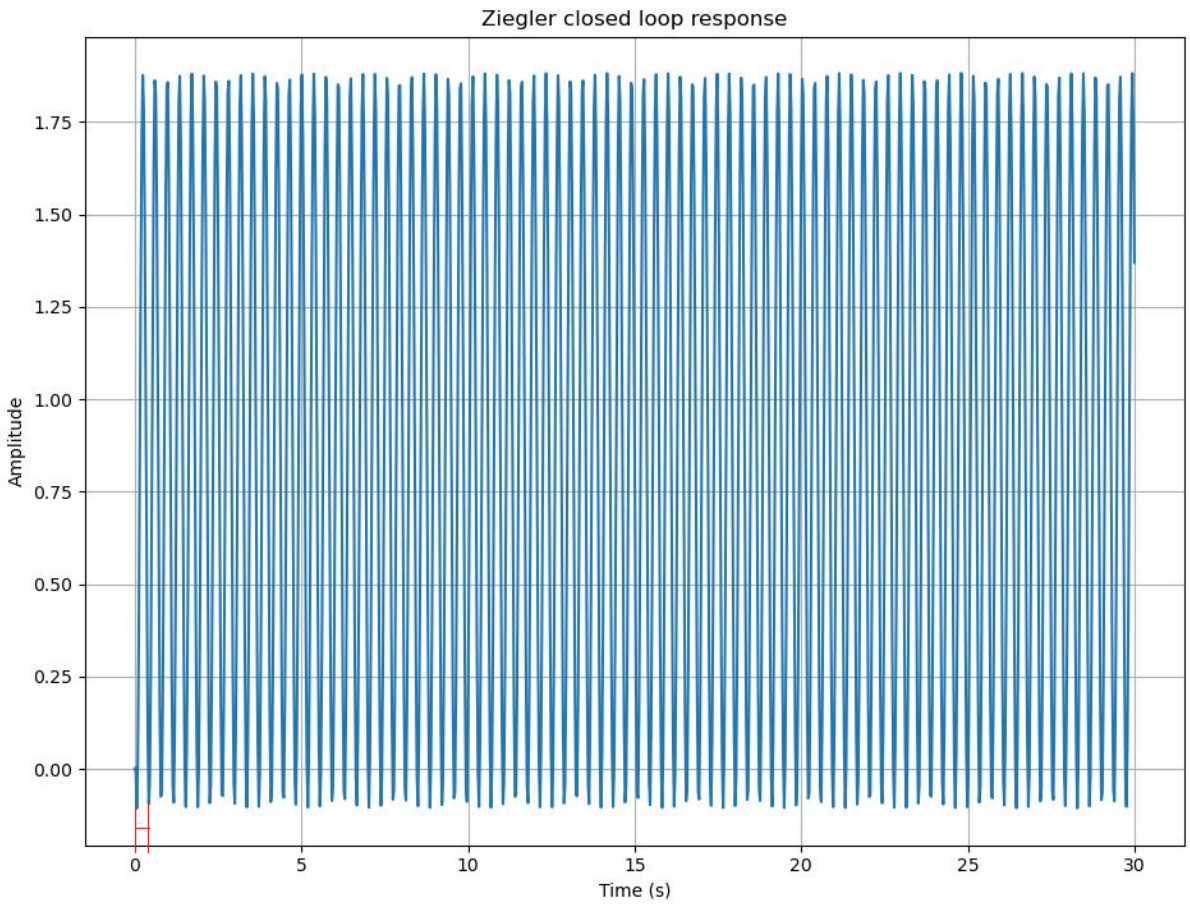

Figure 3.2.3: Closed loop time response of motor plant with ultimate controller gain of 0.09812

This figure is then used to calculate P_u, the period between oscillations. This can be determined by reading the distance between peaks off the graph.

$$P_u = 0.4s$$

These 2 tuning parameters can be used to calculate the parameters of the controller:

$$K_c = 0.6K_{cu}$$
$$K_c = 0.058872$$

$$\tau_1 = 0.5P_u$$
$$\tau_1 = 0.2$$

$$\tau_D = 0.125P_u$$
$$\tau_D = 0.05$$

Therefore the controller is defined as:

$$G_c(s) = K_c(1 + \frac{1}{\tau_1 s} + \tau_D s)$$

$$G_c(s) = 0.058872(1 + \frac{1}{0.2s} + 0.05s)$$

3.3 State-control subsystem controller

The SC^3 will use a PID controller that is tuned experimentally. The offset that was sent by the Sensor Subsystem will be received by the SC^3 and then processed through the PID controller to create a responsive overall system. A random track was generated using a python script to compare the different tuning parameters.
Track code:

```
N = 25
x = np.linspace(0.0, 5.0, 1000)
y = np.random.normal(size = (len(x) + N - 1,)).cumsum() * 10e-3
y = np.convolve(y, np.ones((N,)) / N, mode = 'valid')
y = y - y[0]
def f(xo):
    return np.interp(xo, x, y, left=0)
return f
```

The offset between the middle sensor value and the track is calculated using the following:

```
offset = np.sqrt((x - sens_x) ** 2 + (self.track(x) - sens_y) ** 2)
total_offset += offset
```

The $total_{offset}$ is used to compare how each controller performed.

A proportional only controller will be tested first:

```
sys.r_pwm = int(((np.sign(sys.offset) * -1 *
            (sys.offset ** 2) / (50 ** 2))) / 2 + 0.5) *
            sys.opp_point * 2)
```

This controller had a $total_{offset}$ of $21976mm$ and followed the track successfully.

The next controller is the Discrete Equation PID controller from 1.5:

```
sys.r_pwm = (sys.prev_right_pwm + (-1) *
            (K_c * (sys.offset - sys.prev_offset) +
            ((K_c * h) / T_I) * sys.offset + ((K_c * T_D) / h) *
            (sys.offset - 2 * sys.prev_offset +
            sys.prev_prev_offset)))
```

Before the controller was implemented on the SC^3 micro controller it has been simplified/approximated using the Riemann sum method as follows:

```
P = sys.offset
I += sys.offset * (t - sys.prev_t)
D = (sys.offset - sys.prev_offset) / (t - sys.prev_t)
sys.r_pwm = sys.opp_point + (-1) * (K_c * P + T_I * I + T_D * D)
```

The values K_c, T_I and T_D was successfully derived using experimentation then T_I and T_D was set equal to zero. 0.2 was chosen for K_c. A value bigger than 0.001 for T_D made the system go off track and thus T_D was rather kept as zero. The K_c and T_I tuning parameters were varied and resulted in a max $total_{offset}$ of $75mm$.

4 Simulations

4.1 Motor subsystem simulations

PI tuning using SIMC

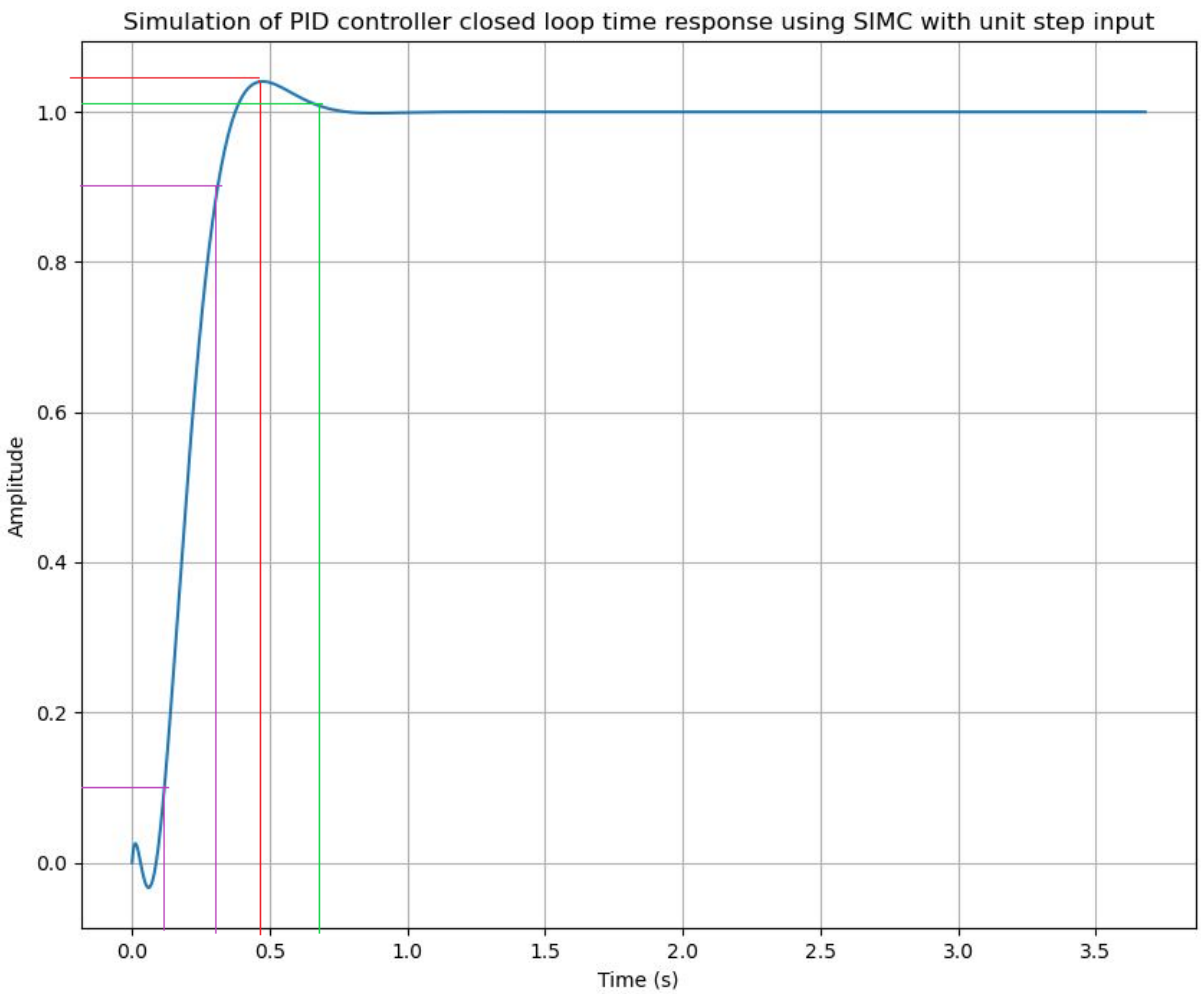

Figure 4.1.1: Simulation of the closed loop response to a step input using the controller approximated with SIMC

As shown above in figure 4.1.1, the simulation of the closed loop time response can be used to calculate the rising, settling and peak times. The Peak time is shown with the red lines, the settling time with the green and the rising time with the difference between the 2 vertical purple lines. As such these performance indicators are as follows:

$$T_s \approx 0.6852s$$
$$T_p \approx 0.4615s$$
$$T_r \approx 0.2963 - 0.1111$$
$$T_r \approx 0.1852s$$

The percentage overshoot can be calculated using the following formula:

$$\%OS = \frac{c_{max} - c_{final}}{c_{final}} \times 100$$
$$\%OS = \frac{1.05 - 1}{1} \times 100$$
$$\%OS = 5$$

Therefore the performance indicators are summarised in the table below:

Table 1: SIMC performance indicators

Performance indicators	
$\%OS$	5 %
T_p	0.4615 s
T_s	0.6852 s
T_r	0.1852 s

Cohen-Coon process reaction curve method

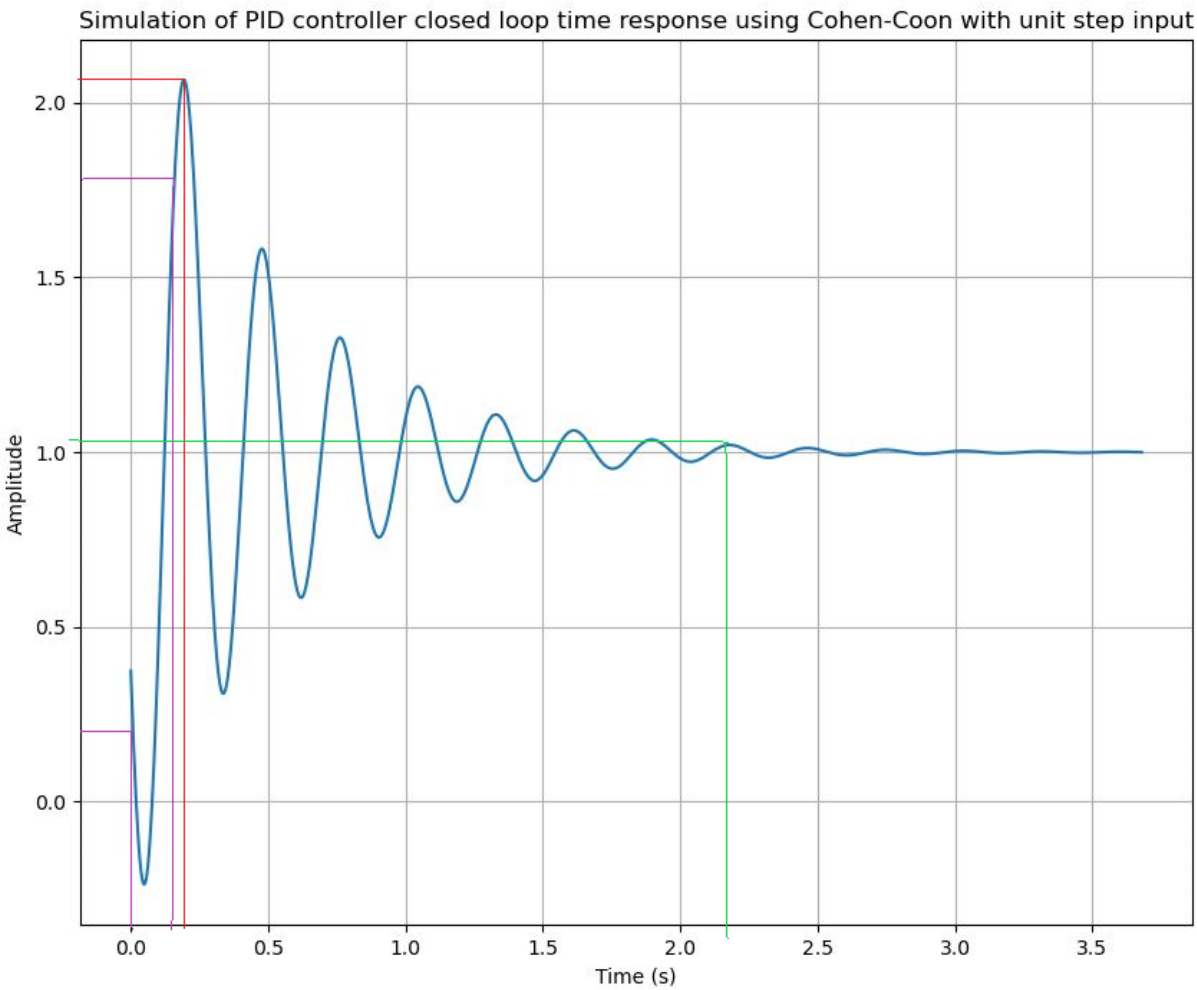

Figure 4.1.2: Simulation of the closed loop response to a step input using the controller approximated with Cohen-Coon

The simulation of the closed loop time response for a controller approximated using the Cohen-Coon process reaction curve method, shown in figure 4.1.2. This figure can be used to determine the rising, settling and peak times. These time follow the previous standard with purple lines indicating rising time, green settling time and red indicating peak time. Therefore:

$$T_s \approx 2.1731s$$

$$T_p \approx 0.1923s$$

$$T_r \approx 0.1538 - 0$$

$$T_r \approx 0.1538s$$

The percentage overshoot can be calculated using the following formula:

$$\%OS = \frac{c_{max} - c_{final}}{c_{final}} \times 100$$

$$\%OS = \frac{2.05 - 1}{1} \times 100$$
$$\%OS = 105$$

The performance indicators are summarised below:

Table 2: Cohen-Coon performance indicators

Performance indicators	
$\%OS$	105 %
T_p	0.1923 s
T_s	2.1731 s
T_r	0.1538 s

Ziegler-Nichols oscillation method

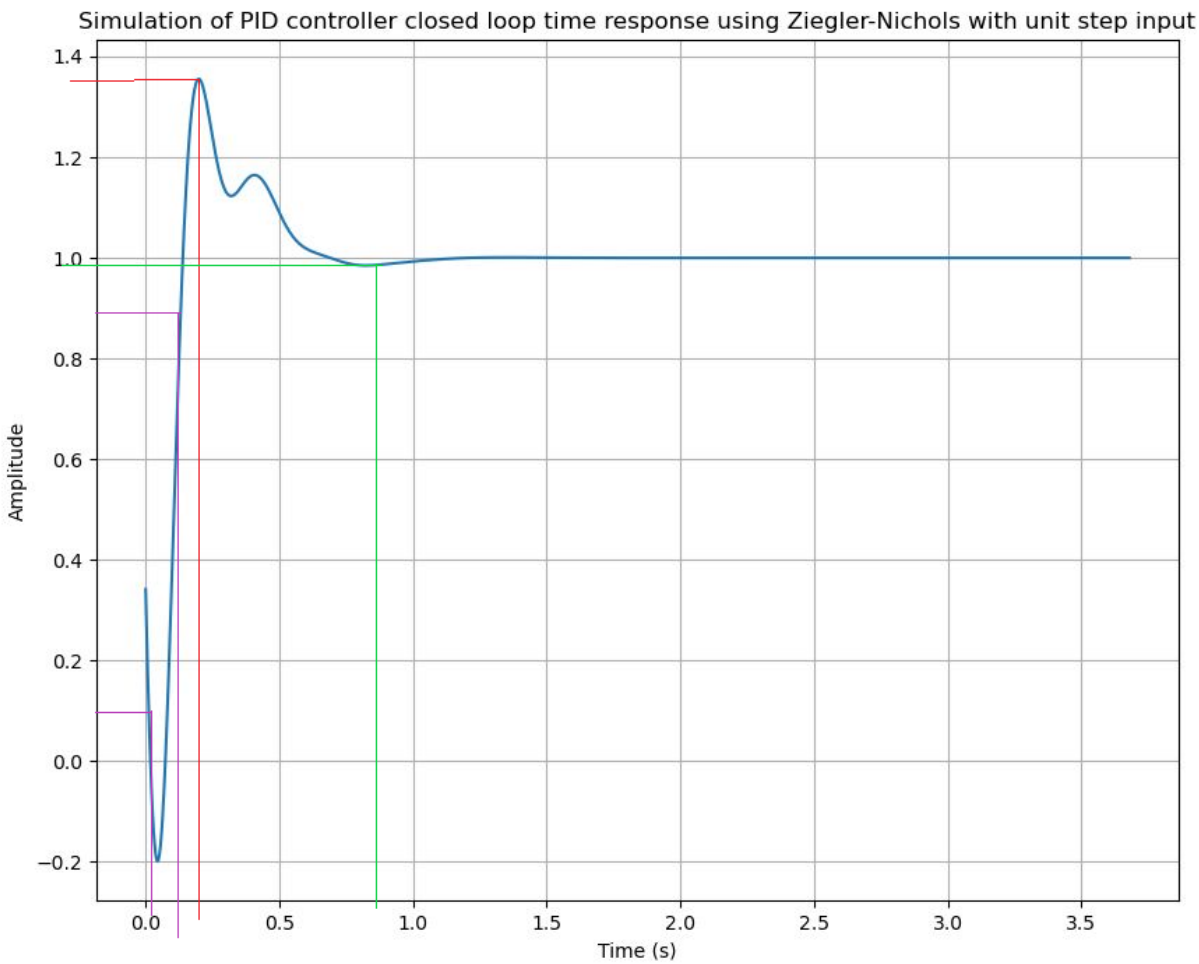

Figure 4.1.3: Simulation of the closed loop response to a step input using the controller approximated with Ziegler-Nichols

The simulation of the closed loop time response for a controller approximated using the Ziegler Nichols oscillation method, shown in figure 4.1.3. This figure can be used to determine the rising, settling and peak times. These time follow the previous standard with purple lines indicating rising time, green settling time and red indicating peak time. Therefore:

$$T_s \approx 0.8704$$

$$T_p \approx 0.1923$$

$$T_r \approx 0.1154 - 0.0192$$

$$T_r \approx 0.0962$$

The percentage overshoot can be calculated using the following formula:

$$\%OS = \frac{c_{max} - c_{final}}{c_{final}} \times 100$$

$$\%OS = \frac{1.35 - 1}{1} \times 100$$

$$\%OS = 35$$

Therefore the performance indicators are summarised in the table below:

Table 3: Ziegler-Nichols performance indicators

Performance indicators	
$\%OS$	35 %
T_p	0.1923 s
T_s	0.8704 s
T_r	0.0962 s

5 Results

5.1 Line-sensor subsystem results

The offset data, as shown below in figure 5.1.1, is generated by reading the generated track using the sensor subsystem. This data is written to a text-file and sent to the S^3C subsystem.

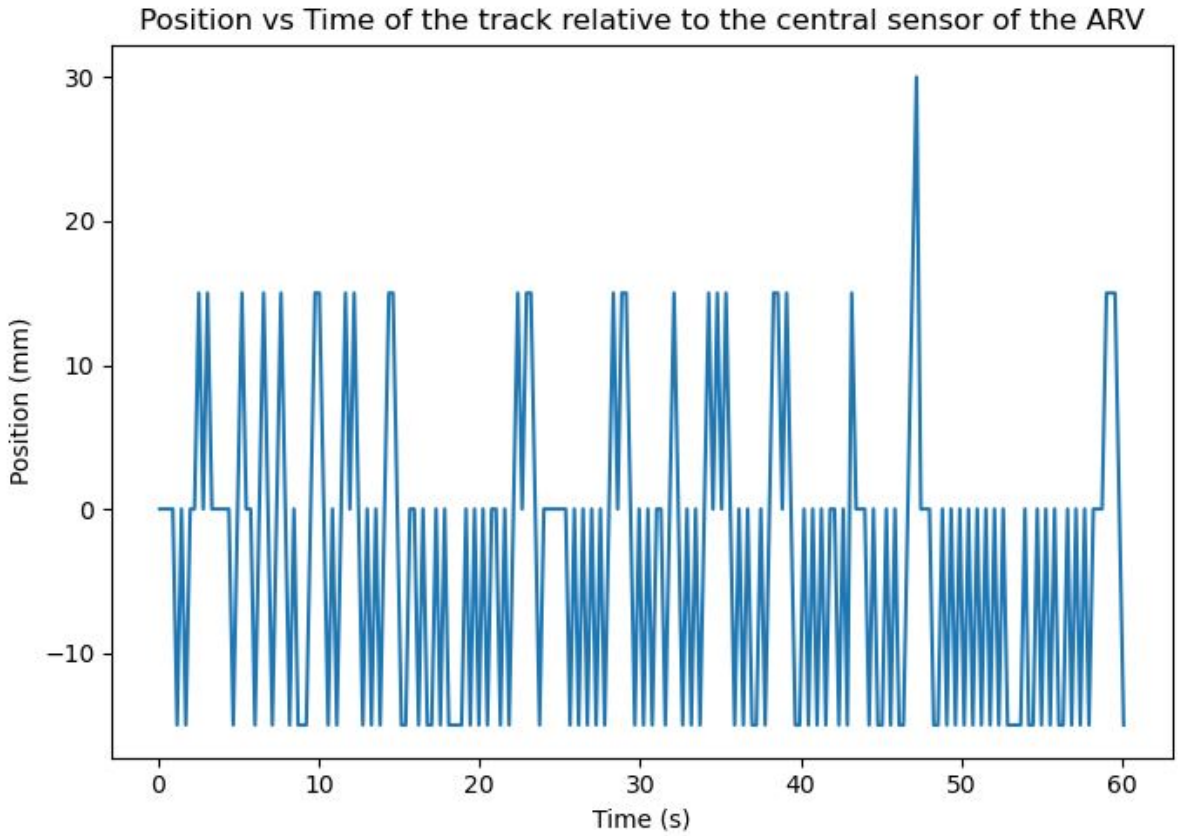

Figure 5.1.1: Line offset vs time plot of the sensor subsystem reading the generated track

5.2 SC^3 subsystem results

The PID Controller for the SC^3 was derived by varying the K_c and T_I to achieve the best, thus lowest $total_{offset}$. The lower the $total_{offset}$ means that the sensor stayed closest to the track. The graph below shows the $total_{offset}$ archived on the generated track with varying K_c and T_I tuning parameter values:

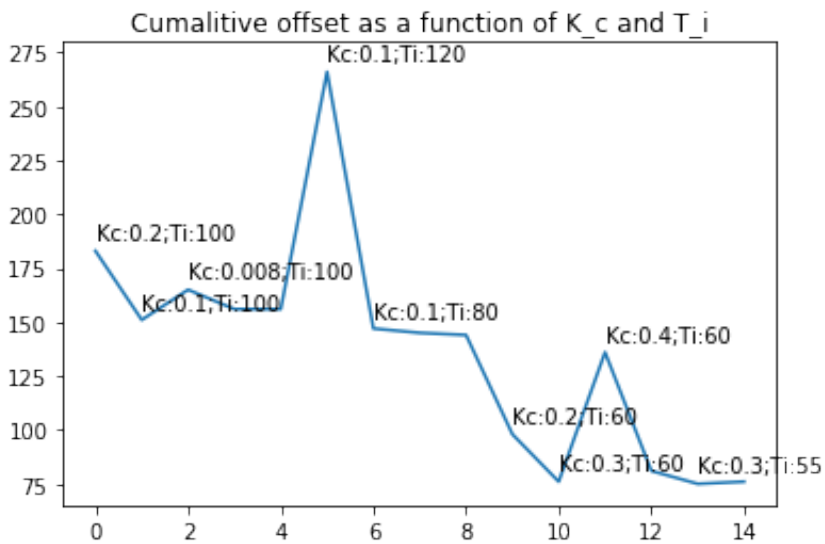

Figure 5.2.1: SC^3 PID Tuning

With the following values, $K_c = 0.3$, $T_I = 55$ and $T_P = 0$ the SC^3 PID controller had a super accurate run with a maximum $total_{offset}$ value of $75mm$ driving on the generated track for 60 seconds.

In real world applications however, the sensor is unable to know the exact location it is away from the track as the sensor can only sense in increments which depend on the size of the sensor. This is why different values were used in the final micro controller's PID Controller, the values chosen were $K_c = 0.2$, $T_I = 0.5$ and $T_P = 0.1$. These values resulted in the following PWM values that is outputted to the motor control subsystem from the sensed values obtained from the sensor subsystem:

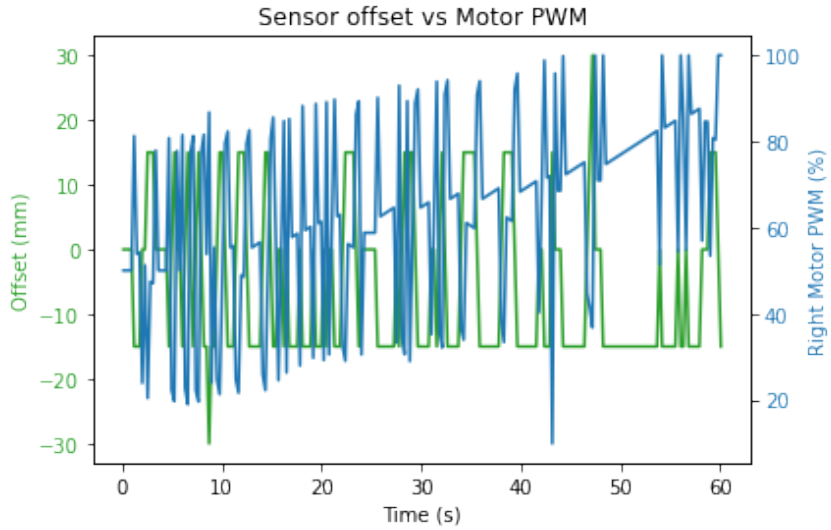

Figure 5.2.2: SC3 PID Practical Tuning

5.3 Motor subsystem results

The PWM output obtained above is used as the PWM input for the motor subsystem. The subsystem then measures the speed and records the time response. The measured speed indicates the movement that the wheel makes to keep the system at the average set value.

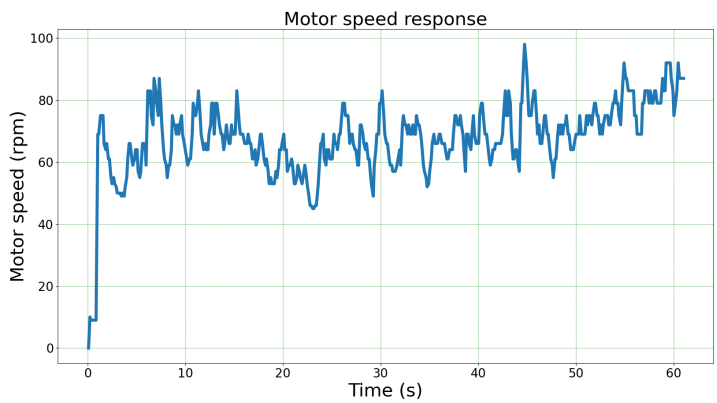

Figure 5.3.1: Motor speed simulation with no PID controller

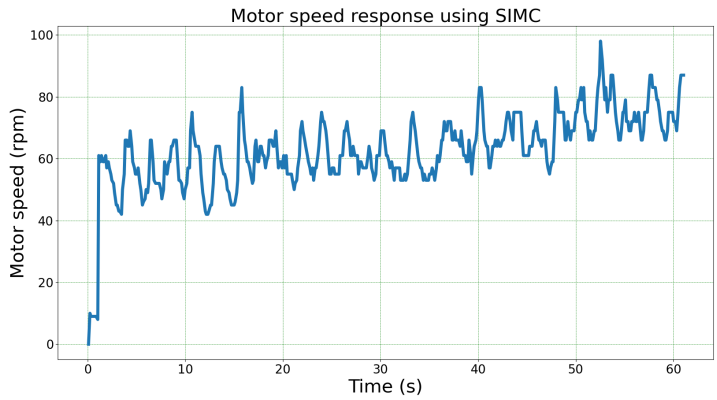

Figure 5.3.2: Motor speed simulation using SIMC PID controller

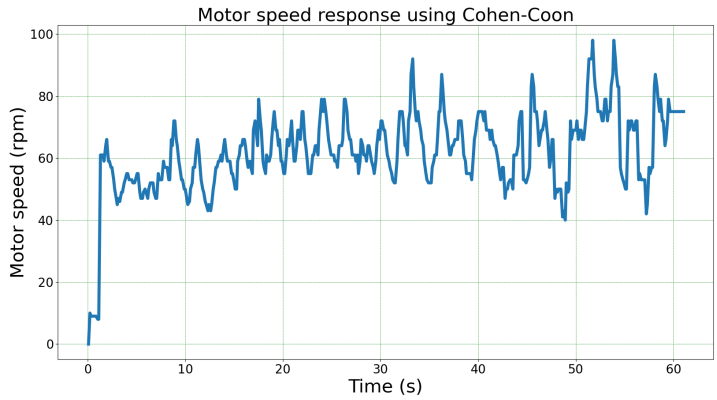

Figure 5.3.3: Motor speed simulation using Cohen-Coon PID controller

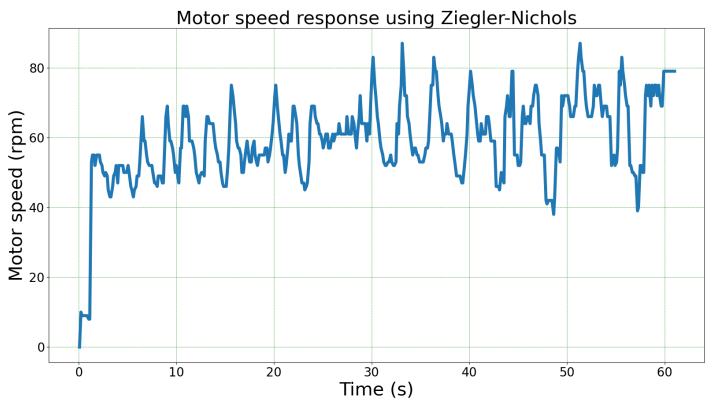

Figure 5.3.4: Motor speed simulation using Ziegler$_N icolsPIDcontroller$

6 Discussion

When comparing the performance indicators of the 3 approximation methods for the PID controller design, one can determine the differences between the approximation methods. The performance indicators are all complied into the table below:

Table 4: Performance indicators of all 3 PID design methods

Performance indicator	**SIMC**	**Cohen-Coon**	**Ziegler-Nichols**
$\%OS$	5%	105 %	35 %
T_p	0.4615 s	0.1923	0.1923 s
T_s	0.6852 s	2.1731	0.8704 s
T_r	0.1852 s	0.1538	0.0962 s

These performance indicators show the difference between the PID design techniques. These allow one to choose which method is most suited for the context. For example, SIMC has the lowest overshoot percentage and settling time, but has the highest peak time and rise time and is thus suited for situations where stability is important, but less suited for context where rise and peak times makes a difference. Cohen-Coon has the highest overshoot and settling times and is thus likely not a valid choice if controller stability is important. It has the lowest peak time and the middle rise time. Ziegler-Nichols has a middling settling time and overshoot but has the lowest rise and peak times.

It is clear that the overshoot and settling time are linked - the method with high overshoot percentages also have high settling times. This is expected as it if a function overshoots it's maximum by a lot it will take longer to converge back to said maximum. In addition to this rise and peak times are also linked - method with high peak times also have high rise times. Once again this result is expected. The rise time is the time taken to rise from 0.1 of the steady-state value to 0.9 of the steady-state value, where peak time is the time taken to rise to the global maxima. These values will be linked as the first ascent (which determines the rise time), also in part determines the peak time as the function peaks and then tends towards it's steady-state value. Therefore a small rise time implies a small peak time.

Ziegler-Nichols and Cohen-Coon appear to both perform well in the same areas (rise and peak times), however Ziegler-Nichols outperforms Cohen-Coon in all performance indicators. Therefore the choice of most suitable controller is between Ziegler-Nichols and SIMC. While both perform very similarly in most indicators SIMc is likely the better choice as it overshoots less and settles faster. The slower rise and peak times are likely less important for the context of an ARV than the settling time and therefore, SIMC is the most appropriate choice for a PID controller approximation method.

7 Conclusion

A PI controller using the SIMC method and two PID controllers using the CohenCoon and Ziegler Nichols methods were designed and simulated using a closed-loop response. Then the actual value of the motor system with 3 different controllers was obtained. Practical and simulations were used as performance indicators. However, there was a clear difference between the simulated and actual values. In the simulated values, the SIMC tuning method showed the worst performance, but in the actual measurements, the performance was the best. Cohen-Coon and Ziegler-Nichols PID-Controllers had a relatively similar values practically and in the simulations.

In addition, a PID controller was developed with experimental tuning for the SC^3. The SC^3 received the line sensor offset values and processed the offsets through the PID controller to produce PWM values. The PWM values was then sent to the motor subsystem asynchronously. The motor's speed was within acceptable margins. This process was repeated using all three of the additional PID controllers, all three responded as expected. The controller adjusted using SIMC responded more smoothly to the offset at a certain time, but the speed of the other two PID controllers was more rough this can be seen from larger spikes in section 5.3.

Overall, SIMC tuning settings was the best controller for practical use. SIMC has the best response time of the three controllers. If there is no differential term, the controller may oscillate, but due to

the nature of the physical system, it will not be carried over to the motor output.

Bibliography

[1] B. Pshenichnyj, "The linearization method," *Optimization*, vol. 18, no. 2, pp. 179–196, 1987.

[2] "Design an arduino based encoder motor using pid controller," Feb 2021. [Online]. Available: https://circuitdigest.com/microcontroller-projects/arduino-based-encoder-motor-using-pid-controller

[3] System, "Arduino uno with rotary encoder to measure rpm," Apr 2014. [Online]. Available: https://forum.arduino.cc/t/arduino-uno-with-rotary-encoder-to-measure-rpm/226182

[4] D. Workshop, "Using rotary encoders with arduino," Sep 2021. [Online]. Available: https://dronebotworkshop.com/rotary-encoders-arduino/

[5] CR7, C. . silver badges66 bronze badges, and C. G. G. . silver badges1212 bronze badges, "Rpm measurement using quadrature encoder," Jun 1966. [Online]. Available: https://arduino.stackexchange.com/questions/49907/rpm-measurement-using-quadrature-encoder

[6] M. B, "Plotting serial port data in real time using python and matplotlib," Feb 2020. [Online]. Available: http://www.mikeburdis.com/wp/notes/plotting-serial-port-data-using-python-and-matplotlib/

[7] S. Fisher, "Circuit diagram web editor," 2013. [Online]. Available: https://www.circuit-diagram.org/editor/

Appendix A

Motor Control Python Code

```
import serial as sr
import matplotlib.pyplot as plt
import numpy as np
import time

def PlotGraph(x,y,title,xlabel,ylabel,fcount,xlimbot=0,xlimtop=0,
ylimbot=0,ylimtop=0,color = 1):
    plt.figure(fcount)
    plt.plot(x,y,color,linewidth=5)
    plt.xlabel(xlabel,fontsize=30)
    plt.ylabel(ylabel,fontsize=30)
    plt.tick_params(labelsize=20)
    plt.title(title,fontsize=30)


    plt.grid(color = 'green', linestyle = '--', linewidth = 0.5)
    if (xlimbot != 0) or (xlimtop != 0):
        plt.xlim(xlimbot,xlimtop)
    if (ylimbot != 0) or (ylimtop != 0):
        plt.ylim(ylimbot,ylimtop)
        fcount += 1
    plt.show()
    return fcount

arduino = sr.Serial('COM6', 19200, timeout = .1)

Counter = 0
F_count = 1

Time = []
RPM = []
Previousrpm = 0

t = [0.123, 0.389, 0.658, 0.9289, 1.198, 1.474, 1.7379, 2.0119, 2.2782, 2.5512, 2.8222,
    3.0932, 3.3621, 3.6351, 3.9031, 4.1481, 4.4161, 4.6901, 4.956, 5.23, 5.5, 5.776,
    6.043, 6.313, 6.5799, 6.8529, 7.1219, 7.3869, 7.6599, 7.9329, 8.199, 8.467, 8.724,
    8.9659, 9.2359, 9.5069, 9.7779, 10.0509, 10.3189, 10.5928, 10.8588, 11.1338, 11.4018,
    11.6718, 11.9438, 12.2108, 12.4791, 12.7501, 13.0201, 13.288, 13.558, 13.829, 14.1,
    14.368, 14.6262, 14.8952, 15.1501, 15.4201, 15.6781, 15.9521, 16.2221, 16.4931,
    16.7621, 17.033, 17.307, 17.572, 17.843, 18.099, 18.374, 18.6389, 18.8949, 19.1679,
    19.4339, 19.7029, 19.9709, 20.2278, 20.5018, 20.7668, 21.0378, 21.3098, 21.579, 21.85,
     22.121, 22.393, 22.66, 22.9329, 23.2059, 23.4739, 23.7459, 24.0109, 24.2849, 24.5588,
     24.825, 25.098, 25.371, 25.6389, 25.9079, 26.1779, 26.4489, 26.7169, 26.9859,
    27.2619, 27.5258, 27.7938, 28.0688, 28.3358, 28.6058, 28.8618, 29.1337, 29.4007,
    29.6697, 29.9377, 30.212, 30.483, 30.754, 31.025, 31.2949, 31.5659, 31.8388, 32.1088,
    32.3758, 32.6438, 32.9158, 33.1845, 33.4515, 33.7265, 33.9925, 34.2645, 34.5315,
    34.8024, 35.0724, 35.3404, 35.6084, 35.8784, 36.1517, 36.4206, 36.6886, 36.9636,
    37.2346, 37.5046, 37.7786, 38.0465, 38.3018, 38.5707, 38.8427, 39.1137, 39.3857,
```

```
    39.6547, 39.9247, 40.1958, 40.4692, 40.7371, 41.0071, 41.2761, 41.5491, 41.8221,
    42.0951, 42.3661, 42.635, 42.892, 43.161, 43.436, 43.708, 43.975, 44.2439, 44.5159,
    44.7879, 45.0579, 45.3139, 45.5839, 45.8528, 46.1078, 46.3808, 46.6538, 46.9258,
    47.1918, 47.4627, 47.7357, 48.0027, 48.2607, 48.5327, 48.8017, 49.0767, 49.3466,
    49.6136, 49.8846, 50.1566, 50.4246, 50.6946, 50.9675, 51.2335, 51.5065, 51.7737,
    52.0427, 52.3127, 52.5827, 52.8577, 53.1247, 53.3946, 53.6706, 53.9376, 54.1776,
    54.4476, 54.7026, 54.9649, 55.2378, 55.5078, 55.7818, 56.0478, 56.3178, 56.5878,
    56.8438, 57.1137, 57.3827, 57.6507, 57.9209, 58.1918, 58.4658, 58.7368, 59.0068,
    59.2748, 59.5408, 59.8067, 60.0757]
pwm = [50, 50.0, 50.0, 50.0, 81.27433004459309, 53.81765, 54.213499999999996,
    23.841237226277354, 51.2135, 20.33147252747254, 47.3975, 46.991, 77.88240944589067,
    49.991, 49.991, 49.991, 49.991, 80.77426277372263, 22.195907484016576,
    19.618737226277375, 77.76877777777781, 49.991, 81.4813876404494, 22.61372222222218,
    18.890737860622018, 77.46367747252751, 81.2756908921933, 22.092763207547133,
    19.512622527472587, 77.45767747252751, 81.56919289740702, 53.7863, 86.74017937743194,
    23.91560266639098, 55.32515, 24.649873247232374, 21.243373247232554, 48.50915,
    79.49422462686579, 82.30225629791883, 55.319, 55.731500000000004, 24.746425373134215,
    21.54872222222218, 48.9185, 48.518, 79.47178307864331, 82.59977675276762,
    55.329499999999996, 55.73135, 56.13635, 56.54285, 25.867573247232375,
    22.1557753731344, 49.75355, 80.6345908921933, 85.55920325617899, 56.54089999999999,
    24.471132558139633, 53.54089999999999, 84.72367777777764, 26.270623247232187,
    85.23044089219312, 57.755750000000006, 58.16675, 58.56425, 27.888973247232556,
    88.24512499999997, 59.360749999999996, 59.7581, 60.1421, 29.669572527472496,
    88.73658872180432, 60.9446, 61.3466, 29.152360996496476, 89.12986277372254,
    30.455713207547234, 58.757600000000004, 89.73912941176486, 62.5694, 62.9759,
    32.300623247232195, 28.9943705882355, 56.1674, 55.758050000000004, 55.34855,
    86.3336246268656, 89.33007941176487, 30.454663207547235, 58.756550000000004,
    58.756550000000004, 58.756550000000004, 58.756550000000004, 58.756550000000004,
    90.15392071668523, 62.56190000000001, 62.96690000000001, 63.373400000000004,
    63.775400000000005, 64.1789, 64.5929, 33.17304399393706, 92.97997462686561,
    34.72217272727259, 30.504512359550514, 89.37217777777784, 28.913525000000032,
    57.802550000000004, 88.8924376404495, 92.08709089219349, 64.60805, 65.0195, 65.426,
    65.83250000000001, 35.157223247232196, 62.8325, 93.91427675276745, 35.756405643092705,
     32.056222222222544, 90.92388764044986, 94.2210746268656, 66.644, 67.04705000000001,
    67.44755, 67.86005, 36.66456127819531, 33.87852058823515, 61.051550000000006, 60.6452,
     60.2402, 59.8382, 90.8232746268656, 94.02097777777819, 66.65315, 67.0565, 67.4585,
    67.871, 68.2775, 68.6825, 69.09349999999999, 38.09797928331513, 33.33334670975303,
    62.307199999999995, 61.89919999999999, 61.49269999999999, 92.06622941176485,
    95.77724089219346, 68.3012, 68.70784999999998, 69.11795, 69.51979999999999,
    69.92479999999999, 70.32829999999998, 70.7378, 40.265272527472135, 67.7378,
    98.81957675276743, 71.54764999999999, 71.93315, 9.76756821561304, 95.80237727272741,
    68.52965, 68.52965, 99.82440944589118, 72.341, 72.749, 73.154, 73.538, 73.943,
    74.34635, 74.72885, 75.13835, 44.66582252747214, 41.156820588235846,
    36.73686127819529, 100, 70.93234999999999, 70.93234999999999, 100, 74.72735, 75.13085,
     75.54335, 75.9482, 76.3487, 76.75519999999999, 77.16319999999999, 77.56519999999999,
    77.97019999999999, 78.37955, 78.77855, 79.18804999999999, 79.58884999999998,
    79.99234999999999, 80.39734999999999, 80.80234999999999, 81.21485, 81.61534999999999,
    82.02019999999999, 82.43419999999999, 51.34431235955087, 100, 83.19919999999999,
    83.58169999999998, 83.97514999999999, 84.38449999999999, 84.7895, 54.417237226276754,
    100, 85.5935, 54.81572222222255, 100, 86.38235, 86.78585, 87.18785, 87.59315000000001,
     56.907657198227874, 84.59315000000001, 84.59315000000001, 53.41037222222181,
    80.78614999999999, 80.38715, 100, 100]


arduino.close()
arduino.open()

time.sleep(2)

def Updaterpm():
    arduino.write(bytes("[162,10,90,0]", 'utf-8'))
    temp = arduino.readline().decode('utf-8')
    helper = 0
    toconvert = ""
    for i in range(len(temp)):
        if(temp[i] == ","):
```

```
            helper+=1
        elif(helper == 1):
            toconvert += temp[i]
    RecInt = int(toconvert)
    RPM.append(RecInt)
    Time.append(CTime - STime)
    #print("Time: " + str(round(CTime - STime,2)) + " RPM: " + str(RecInt))


TransPacketSend = False
time.sleep(5)

STime = time.time()
htime = time.time()
CTime = STime
Counter = 0

while (CTime - STime) < 61 :
    CTime = time.time()

    if CTime - htime > 0.1:

        try:
            Updaterpm()
        except:
            continue
        htime = CTime


    if ((CTime - STime) >= t[Counter]) :
        helper = pwm[Counter]
        if Counter < len(t)-1:
            Counter += 1
        print("Time : ", helper)
        helperstring = "[161,"+str(helper)+",0,0]"
        arduino.write(bytes(helperstring, 'utf-8'))
        try:
            Updaterpm()
        except:
            continue

arduino.close()

F_count = PlotGraph(Time, RPM,
"Motor speed response using Ziegler-Nichols",
 "Time (s)","Motor speed (rpm)", F_count)

# def write_read(x):
#     arduino.write(bytes(x, 'utf-8'))
#     time.sleep(1)
#     data = arduino.readline()
#     return data
# while True:
#     num = input("Data packet sent to Arduino: ")
#     value = write_read(num)
#     print("Data packet recieved from Arduino: ",value)
```

Appendix B

Motor Control Arduino code

```
#define PWM 6

#define ENCA 2
#define ENCODEROUTPUT 350

long encoderValue = 0;
long pulsesIN = 0;
int rotations = 0;

long previousMillis = 0;
long currentMillis = 0;
long deltatime = 0;


int rpm = 0;
boolean measureRpm = false;
bool PWMwait;

long Right_pulse_time[] =
    {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0};

int pwm_val = 0;
String Serial_data;
String helper1;
String helper2;
String helper3;
String helper4;



void setup() {
  // put your setup code here, to run once:
  Serial.begin(19200);

  Serial.setTimeout(14);

  pinMode(PWM, OUTPUT);

  pinMode(ENCA, INPUT);

  attachInterrupt(digitalPinToInterrupt(ENCA), updateEncoder, CHANGE);

  previousMillis = millis();

  PWMwait = false;
}

void loop() {
```

```
  // put your main code here, to run repeatedly:

  while(Serial.available() > 0 ){
    Serial_data += Serial.readString();
  }

  interSerial();

  if(pwm_val > 255)
    pwm_val = 0;


  Serial_data = "";

}


void updateRPM()
{
  previousMillis = currentMillis;
  rpm = (long)(encoderValue * 60 / 360);
  encoderValue = 0;

}

void interSerial()
{
  int counter = 0;
  helper1 = "";
  helper2 = "";
  helper3 = "";
  helper4 = "";
  for(int i=0; i < Serial_data.length(); i++){
    if(Serial_data[i] == '[')
      continue;

    else if(Serial_data[i] == ']')
      continue;

    else if(Serial_data[i] == ',')
      counter++;

    else{
      if (counter == 0)
        helper1 += Serial_data[i];
      else if (counter == 1)
        helper2 += Serial_data[i];
      else if (counter == 2)
        helper3 += Serial_data[i];
      else if (counter == 3)
        helper4 += Serial_data[i];
    }
  }

  if(helper1.toInt() == 161){
    int x = 0;
    float z = 0;
    x = helper2.toInt();
    float temp_pwm = 0.0;
    z = (float)x/100;
    temp_pwm =78.0+ z*177.0;
    /*SIMC
    temp_pwm = ((1.0+0.46*temp_pwm)/16.20*temp_pwm)*/
    /*Cohen-Coon
    temp_pwm = temp_pwm + 0.0964*(1.0+1.0/(0.2543*temp_pwm)+0.352*temp_pwm);*/

```

```
    temp_pwm = temp_pwm + 0.058872*(1.0+1.0/(0.2*temp_pwm)+0.5*temp_pwm);

    pwm_val = temp_pwm + 0.5;
    analogWrite(PWM, pwm_val);
    PWMwait = true;
  }



  else if(helper1.toInt() == 162){
    String Tosend = "";
    int rpm_overshoot = 0;
    rpm_overshoot = rpm;
    if(rpm_overshoot > 130)
      rpm_overshoot = 130;
    Tosend = "[163," + String(rpm_overshoot) + ","+ String(130 - rpm_overshoot) + "," + 0
    + "]";
    Serial.println(Tosend);
  }
}

void updateEncoder()
{
  for(int i = 35; i >= 0; i--)
  {
    if(i == 0)
    {
      Right_pulse_time[i] = millis();
    }
    else
    {
      Right_pulse_time[i] = Right_pulse_time[i-1];
    }
  }
  long avg = 0.0;
  for(int i = 0; i < 35; i++)
  {
    avg += Right_pulse_time[i] - Right_pulse_time[i+1];
  }
  long temp_avg = avg/35.0;
  long temp_R_rpm = 60000/(temp_avg * 36.0);
  rpm = temp_R_rpm + 0.5;
  encoderValue++;

}
```

Appendix C

Controller design and simulation python code

```python
# Author: Timothy Fogwill
# EBB practical 3
# Last Update: 2021/11/06

import math
import time
import serial
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import control as ct

s = ct.TransferFunction.s


def plot(x, y, title, x_label, y_label):
    plt.figure(figsize = (10, 8))
    plt.title(title)
    plt.ylabel(y_label)
    plt.xlabel(x_label)
    plt.grid()
    plt.plot(x, y)
    plt.show()

def plotBode(x, y, x1, y1, title):
    fig, axs = plt.subplots(2)
    fig.suptitle(title)

    axs[0].semilogx(x, y)
    axs[0].set(xlabel = "frequency (rad/s)", ylabel = "Magnitude")
    axs[1].semilogx(x1, y1)
    axs[1].set(xlabel = "frequency (rad/s)", ylabel = "Phase (rad)")
    fig.show()

def plotMotorPlant():
    K = 81
    t_d = 0.1
    t_p = 0.46

    # approximate e^(-t_d s) using (2, 2) Pade approximate. Accurate for values close to 0
    x = -1*t_d*s
    exp_val = ((x + 3)*(x + 3) + 3)/((x - 3)*(x - 3) + 3)

    G_M = (K * exp_val)/(t_p*s + 1)

```

```
    t = np.linspace(0, 10*t_p, 1000)

    t, val = ct.step_response(G_M, t)


    in_title = "Motor plant open loop time response to unit step input"
    x_lab = "Time (s)"
    y_lab = "Amplitude"
    plot(t, val, in_title, x_lab, y_lab)

def simController(Kc, t1, td, method):
    K = 81
    t_d = 0.1
    t_p = 0.46
    # approximate e^(-t_d s) using (2, 2) Pade approximate. Accurate for values close to 0
    x = -1*t_d*s
    exp_val = ((x + 3)*(x + 3) + 3)/((x - 3)*(x - 3) + 3)

    G_M = (K * exp_val)/(t_p*s + 1)
    G_c = Kc*(1 + 1/(t1*s) + td*s)

    G = G_M * G_c

    H = (G)/(1 + G)
    t = np.linspace(0, 8*t_p, 1000)

    t, val = ct.step_response(H, t)

    in_title = "Simulation of PID controller closed loop time response using " + method + " with unit step input"
    x_lab = "Time (s)"
    y_lab = "Amplitude"
    plot(t, val, in_title, x_lab, y_lab)


def ZieglerTest(K_c):
    #Motor plant
    K = 81
    t_d = 0.1
    t_p = 0.46

    # approximate e^(-t_d s) using (2, 2) Pade approximate. Accurate for values close to 0
    x = -1*t_d*s
    exp_val = ((x + 3)*(x + 3) + 3)/((x - 3)*(x - 3) + 3)
    G_M = K_c*(K * exp_val)/(t_p*s + 1)


    x_1 = 1000
    x_2 = 0.001
    G_c = K_c*(1 + 1/(x_1 * s) + x_2 * s)

    t = np.linspace(0, 30, 1000)

    G = G_M

    H = G/(1 + G)
    t, val = ct.step_response(H, t)

    # Bode
    Bode_eq = G_M
    magnitude, phase, freq = ct.bode(Bode_eq)
    plotBode(freq, magnitude, freq, phase, "Bode plot for Ziegler-Nichols PID open loop response")


    # closed loop
```

```

    in_title = "Ziegler closed loop response"
    x_lab = "Time (s)"
    y_lab = "Amplitude"
    plot(t, val, in_title, x_lab, y_lab)

#plotMotorPlant()
#ZieglerTest(0.09812)

#simController(0.0284, 0.46, 0, "SIMC")
#simController(0.0964, 0.2543, 0.0352, "Cohen-Coon")
simController(0.058872, 0.2, 0.05, "Ziegler-Nichols")
```

Appendix D

Line sensor Arduino code

```
// Author: Timothy Fogwill
// EBB prac 2 code
// Last Update: 06/10/2021

int controlNum = 0;
int recPacket[4] = {0, 0, 0, 0};
bool received = false;
int packet[4] = {0, 0, 0, 0};

int directionOut = 0;
int deviationOut = 0;

// Anything less than 308*1024/5 + 205/1024*5 = 1.5 + 1  = 2.5V
// is considered green track. Anything above that is white
int greenVolLevel = 308;
int voltBuffer = 205;

int threshold = greenVolLevel + voltBuffer;

// will be read using analogRead - ADC values
int sensorData[5] = {0, 0, 0, 0, 0};

void setup() {
  Serial.begin(19200);
}

void loop() {
  delay(0.1);
  // Packet has not been read
  if (Serial.available() > 0) {
    recPacket[controlNum] = Serial.read();

    if (controlNum != 3){
      controlNum++;
      }
    else {
      for (int i = 0; i < 4; i++){
        packet[i] = recPacket[i];
        }
      controlNum = 0;
      received = true;
      }
    }
  else { // Packet has been read
    if ((packet[0] == 163) and (received == true)) {
      received = false;

      // Read voltage values into ADC int vars
```

```
      // Top sensor is connected tp A0, botttom to A4
      sensorData[0] = analogRead(A0);
      sensorData[1] = analogRead(A1);
      sensorData[2] = analogRead(A2);
      sensorData[3] = analogRead(A3);
      sensorData[4] = analogRead(A4);

      int dummyVar = 10;
      for (int i = 0; i < 5; i++) {
        if (sensorData[i] < threshold) {
          dummyVar = i;

         // if sensor detects line on more than 1  sensor, favour sensor that causes
         // less turning - less destructive if its wrong. Therefore most readings will
         // be centre or slight turns with sharp turns beig rare (and only used if absolutely needed).

          if ((i == 1) or (i == 3)) {
            break;
            }
          }
        }

      switch(dummyVar) {
        case 0:
          deviationOut = 30;
          directionOut = 2;
          break;
        case 1:
          deviationOut = 15;
          directionOut = 2;
          break;
        case 2:
          deviationOut = 0;
          directionOut = 0;
          break;
        case 3:
          deviationOut = 15;
          directionOut = 0;
          break;
        case 4:
          deviationOut = 30;
          directionOut = 0;
          break;
        default:
          deviationOut = 0;
          directionOut = 0;
          break;
        }

      sendData(177, deviationOut, 0, directionOut);
      }
    }
}

void sendData(int b1, int b2, int b3, int b4) {
  Serial.write(b1);
  Serial.write(b2);
  Serial.write(b3);
  Serial.write(b4);
  }
```

Appendix E

Line sensor Python code

```python
# Author: Timothy Fogwill
# EBB prac 2 logging code
# Last Update: 06/10/2021


import serial
import time
import matplotlib.pyplot as plt

filename = "PositionFie.txt"

def plotPosfile():
    file = open(filename, "r")
    time = []
    pos = []
    Lines = file.readlines()

    for line in Lines:
        hold = int(line.find(',', 0, -1))

        time.append(float(line[0 : hold]))
        pos.append(int(line[hold + 1:]))


    plt.title("Position vs Time of the track relative to the central sensor of the ARV")
    plt.ylabel("Position (mm)")
    plt.xlabel("Time (s)")
    plt.plot(time, pos)
    plt.show()
    file.close()


def testlineSensor():

    ser = serial.Serial(port = "COM5", baudrate = 19200)

    ser.close()
    ser.open()

    file = open(filename, "w")

    time.sleep(2)

    bSent = False
    startTime = time.time()
    currTime = time.time()

    localTime = currTime - startTime
```

```python

    while (localTime < 60):
        currTime = time.time()
        localTime = currTime - startTime


        if (bSent == False):
            time.sleep(0.1)
            ser.write(bytes([163]))
            ser.write(bytes([0]))
            ser.write(bytes([0]))
            ser.write(bytes([0]))
            bSent = True
            print("At " + str(round(localTime, 4)) + "s - Sent: 163, 0, 0, 0")

        elif ((ser.inWaiting() > 0) and (bSent == True)):
            controlNum = 0
            packet = [0, 0, 0, 0]
            bReceived = False


            while (bReceived == False):
                if (ser.inWaiting()):
                    hold = ser.read()

                    packet[controlNum] = int.from_bytes(hold, 'big') # try little if doesn't work
                    controlNum = controlNum + 1

                    if (controlNum > 3):
                        breceived = True
                        break

            print("At " + str(round(localTime, 4)) + "s - Received: " + str(packet[0]) + ', ' + str(packet[1]) + ', ' +
                    str(packet[2]) + ', ' + str(packet[3]))


            posVal = packet[1]
            if packet[3] == 2:
                posVal = packet[1] * -1

            file.write(str(round(localTime, 4)) + "," + str(posVal) + "\n")

            bSent = False
            time.sleep(0.1)

    file.close()
    ser.close()
```